

Lab 3 Tutorial: User Input, State Management, and Navigation

This comprehensive tutorial covers essential React Native concepts for building interactive mobile applications:

- 1. **Props and States** - Understanding component data flow
 - 2. **React Hooks (useState)** - Managing component state with hooks
 - 3. **Handling User Input & Forms** - Working with TextInput and form validation
 - 4. **Handling Touches** - Responding to user touch interactions
 - 5. **Introduction to Navigation** - Overview of Stack, Tab, and Drawer navigators
 - 6. **Navigating Between Screens** - Implementing navigation in your app
 - 7. **Passing Data Using Navigation** - Sharing data between screens
-

1. Props and States

Understanding the difference between **props** and **state** is fundamental to React and React Native.

What are Props?

Props (Properties) are data passed from a parent component to a child component. Props are **immutable** (read-only) and cannot be changed by the child component.

What is State?

State is data that belongs to a component and can change over time. When state changes, the component re-renders to reflect the new data.

Key Differences

Aspect	Props	State
Source	Passed from parent	Managed within component
Mutability	Immutable (read-only)	Mutable (can be changed)
Scope	Component receives props	Component owns state
Updates	Parent component changes props	Component updates state using useState setter

Props Example

```
import { View, Text, StyleSheet } from 'react-native';

// Parent Component
function App() {
  return (
```

```
    <View>
      <Greeting name="Alice" age={25} />
      <Greeting name="Bob" age={30} />
    </View>
  );
}

// Child Component (receives props)
function Greeting({ name, age }) {
  return (
    <View style={styles.container}>
      <Text>Hello, {name}!</Text>
      <Text>You are {age} years old.</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    padding: 10,
  },
});

export default App;
```

2. React Hooks (useState)

Hooks are functions that let you "hook into" React features like state and lifecycle methods from functional components. `useState` is the most commonly used hook.

Why Hooks?

- ☒ Simpler syntax than class components
- ☒ Better code reusability
- ☒ Recommended by React team
- ☒ Works with functional components

useState Hook

`useState` returns an array with two elements:

1. The current state value
2. A function to update the state

Basic useState Syntax

```
import { useState } from 'react';

const [stateValue, setStateValue] = useState(initialValue);
```

useState Example - Counter

```
import { useState } from 'react';
import { View, Text, Button, StyleSheet } from 'react-native';

function Counter() {
  const [count, setCount] = useState(0);

  const increment = () => {
    setCount(count + 1);
  };

  const decrement = () => {
    setCount(count - 1);
  };

  const reset = () => {
    setCount(0);
  };

  return (
    <View style={styles.container}>
      <Text style={styles.text}>Count: {count}</Text>
      <View style={styles.buttonContainer}>
        <Button title="Decrement" onPress={decrement} />
        <Button title="Reset" onPress={reset} />
        <Button title="Increment" onPress={increment} />
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 20,
  },
  text: {
    fontSize: 32,
    fontWeight: 'bold',
    marginBottom: 20,
  },
  buttonContainer: {
    flexDirection: 'row',
    gap: 10,
  },
});
```

Multiple State Variables

You can use multiple `useState` hooks in a single component:

```
import { useState } from 'react';
import { View, Text, StyleSheet } from 'react-native';

function UserProfile() {
  const [name, setName] = useState('');
  const [age, setAge] = useState(0);
  const [email, setEmail] = useState('');

  return (
    <View style={styles.container}>
      <Text>Name: {name || 'Not set'}</Text>
      <Text>Age: {age}</Text>
      <Text>Email: {email || 'Not set'}</Text>
    </View>
  );
}
```

State Updates with Functions

When the new state depends on the previous state, use a function:

```
// ✓ Correct - Uses previous state
setCount((prevCount) => prevCount + 1);

// ✓ Also correct for simple updates
setCount(count + 1);

// ✗ Incorrect - Don't do this for multiple rapid updates
setCount(count + 1);
setCount(count + 1); // This doesn't work as expected
```

Important Notes about useState

- State updates are **asynchronous**
- Calling `setState` triggers a **re-render**
- State is **component-specific** (each component instance has its own state)
- Use **functional updates** when state depends on previous state

3. Handling User Input & Forms

React Native provides `TextInput` component for handling user text input. Forms are created by combining multiple input fields with state management.

TextInput Component

The `TextInput` component is similar to HTML's `<input>` element but adapted for mobile.

Basic TextInput

```
import { useState } from 'react';
import { View, Text, TextInput, StyleSheet } from 'react-native';

function BasicInput() {
  const [text, setText] = useState('');

  return (
    <View style={styles.container}>
      <TextInput
        style={styles.input}
        placeholder="Enter your name"
        value={text}
        onChangeText={setText}
      />
      <Text style={styles.displayText}>
        You typed: {text}
      </Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    padding: 20,
  },
  input: {
    height: 40,
    borderWidth: 1,
    borderColor: '#ccc',
    borderRadius: 5,
    paddingHorizontal: 10,
    marginBottom: 10,
  },
  displayText: {
    fontSize: 16,
    marginTop: 10,
  },
});
```

Common TextInput Props

Prop	Type	Description
value	string	The value of the text input (controlled component)

Prop	Type	Description
<code>onChangeText</code>	function	Callback when text changes
<code>placeholder</code>	string	Placeholder text shown when input is empty
<code>placeholderTextColor</code>	string	Color of placeholder text
<code>secureTextEntry</code>	boolean	Hide text (for passwords)
<code>keyboardType</code>	string	Type of keyboard (default, email-address, numeric, etc.)
<code>autoCapitalize</code>	string	Capitalization behavior (none, sentences, words, characters)
<code>autoCorrect</code>	boolean	Enable/disable autocorrect
<code>multiline</code>	boolean	Allow multiple lines of text
<code>editable</code>	boolean	Whether the input is editable
<code>maxLength</code>	number	Maximum number of characters

TextInput with Different Keyboard Types

```
import { useState } from 'react';
import { View, Text, TextInput, StyleSheet } from 'react-native';

function InputExample() {
  const [email, setEmail] = useState('');
  const [phone, setPhone] = useState('');
  const [password, setPassword] = useState('');

  return (
    <View style={styles.container}>
      <TextInput
        style={styles.input}
        placeholder="Email"
        keyboardType="email-address"
        autoCapitalize="none"
        value={email}
        onChangeText={setEmail}
      />

      <TextInput
        style={styles.input}
        placeholder="Phone Number"
        keyboardType="phone-pad"
        value={phone}
        onChangeText={setPhone}
      />

      <TextInput
        style={styles.input}
        placeholder="Password"

```

```

        secureTextEntry={true}
        value={password}
        onChangeText={setPassword}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    padding: 20,
  },
  input: {
    height: 50,
    borderWidth: 1,
    borderColor: '#ccc',
    borderRadius: 8,
    paddingHorizontal: 15,
    marginBottom: 15,
    fontSize: 16,
  },
});

```

Complete Form Example

```

import { useState } from 'react';
import {
  View,
  Text,
  TextInput,
  Button,
  StyleSheet,
  Alert,
  ScrollView,
} from 'react-native';

function LoginForm() {
  const [formData, setFormData] = useState({
    email: '',
    password: '',
  });

  const handleInputChange = (field, value) => {
    setFormData((prev) => ({
      ...prev,
      [field]: value,
    }));
  };

  const handleSubmit = () => {
    if (!formData.email || !formData.password) {

```

```

    Alert.alert('Error', 'Please fill in all fields');
    return;
  }

  if (!formData.email.includes('@')) {
    Alert.alert('Error', 'Please enter a valid email address');
    return;
  }

  Alert.alert('Success', `Welcome, ${formData.email}!`);
  // Reset form
  setFormData({ email: '', password: '' });
};

return (
  <ScrollView style={styles.container}>
    <Text style={styles.title}>Login</Text>

    <Text style={styles.label}>Email</Text>
    <TextInput
      style={styles.input}
      placeholder="Enter your email"
      keyboardType="email-address"
      autoCapitalize="none"
      value={formData.email}
      onChangeText={(value) => handleInputChange('email', value)}
    />

    <Text style={styles.label}>Password</Text>
    <TextInput
      style={styles.input}
      placeholder="Enter your password"
      secureTextEntry={true}
      value={formData.password}
      onChangeText={(value) => handleInputChange('password', value)}
    />

    <Button title="Login" onPress={handleSubmit} />
  </ScrollView>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    backgroundColor: '#fff',
  },
  title: {
    fontSize: 28,
    fontWeight: 'bold',
    marginBottom: 30,
    textAlign: 'center',
  },
});

```

```
label: {
  fontSize: 16,
  fontWeight: '600',
  marginBottom: 8,
  marginTop: 15,
},
input: {
  height: 50,
  borderWidth: 1,
  borderColor: '#ddd',
  borderRadius: 8,
  paddingHorizontal: 15,
  fontSize: 16,
  backgroundColor: '#f9f9f9',
},
});
```

Form Validation

```
function validateForm(email, password) {
  const errors = {};

  if (!email) {
    errors.email = 'Email is required';
  } else if (!/\S+@\S+\.\S+/.test(email)) {
    errors.email = 'Email is invalid';
  }

  if (!password) {
    errors.password = 'Password is required';
  } else if (password.length < 6) {
    errors.password = 'Password must be at least 6 characters';
  }

  return errors;
}

// Usage in component
const handleSubmit = () => {
  const errors = validateForm(formData.email, formData.password);

  if (Object.keys(errors).length > 0) {
    // Display errors
    setFormErrors(errors);
    return;
  }

  // Proceed with form submission
};
```

4. Handling Touches

React Native provides several components for handling touch interactions: `TouchableOpacity`, `TouchableHighlight`, `TouchableWithoutFeedback`, and `Pressable`.

TouchableOpacity

Most commonly used touch component. Provides visual feedback by reducing opacity when pressed.

```
import { TouchableOpacity, Text, StyleSheet } from 'react-native';

function TouchableButton() {
  const handlePress = () => {
    console.log('Button pressed!');
  };

  return (
    <TouchableOpacity
      style={styles.button}
      onPress={handlePress}
      activeOpacity={0.7}
    >
      <Text style={styles.buttonText}>Press Me</Text>
    </TouchableOpacity>
  );
}

const styles = StyleSheet.create({
  button: {
    backgroundColor: '#007AFF',
    padding: 15,
    borderRadius: 8,
    alignItems: 'center',
  },
  buttonText: {
    color: 'fff',
    fontSize: 16,
    fontWeight: '600',
  },
});
```

TouchableHighlight

Similar to `TouchableOpacity` but changes background color instead of opacity.

```
import { TouchableHighlight, Text, StyleSheet } from 'react-native';

function HighlightButton() {
  return (
    <TouchableHighlight
```

```

        style={styles.button}
        onPress={() => console.log('Pressed')}
        underlayColor="#0056CC"
      >
        <Text style={styles.buttonText}>Highlight Button</Text>
      </TouchableHighlight>
    );
  }
}

```

Pressable (Recommended)

Pressable is the most flexible and modern approach. It provides more control over press behavior.

```

import { Pressable, Text, StyleSheet } from 'react-native';

function CustomButton() {
  return (
    <Pressable
      style={({ pressed }) => [
        styles.button,
        pressed && styles.buttonPressed,
      ]}
      onPress={() => console.log('Pressed')}
    >
      <Text style={styles.buttonText}>Pressable Button</Text>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  button: {
    backgroundColor: '#007AFF',
    padding: 15,
    borderRadius: 8,
    alignItems: 'center',
  },
  buttonPressed: {
    backgroundColor: '#0056CC',
    opacity: 0.8,
  },
  buttonText: {
    color: 'fff',
    fontSize: 16,
    fontWeight: '600',
  },
});

```

Long Press Example

```
import { Pressable, Text, StyleSheet, Alert } from 'react-native';

function LongPressButton() {
  return (
    <Pressable
      style={styles.button}
      onPress={() => Alert.alert('Single Press')}
      onLongPress={() => Alert.alert('Long Press')}
    >
      <Text style={styles.buttonText}>
        Press or Long Press Me
      </Text>
    </Pressable>
  );
}
```

Touch Events Comparison

Component	Visual Feedback	Use Case
TouchableOpacity	Opacity change	Buttons, links
TouchableHighlight	Background color change	List items, cards
TouchableWithoutFeedback	None	Custom feedback needed
Pressable	Customizable	Most flexible option

5. Introduction to Navigation

Navigation allows users to move between different screens in your app. React Navigation is the most popular navigation library for React Native.

Types of Navigators

1. Stack Navigator

Navigates between screens like a stack (last-in-first-out). Common for hierarchical navigation.

Characteristics:

- Push new screens on top
- Pop screens to go back
- Shows header with back button
- Good for: Detail screens, forms, authentication flows

2. Tab Navigator

Shows multiple screens with tabs at the bottom or top of the screen.

Characteristics:

- All tabs visible at once
- Easy switching between tabs
- Icons and labels for each tab
- Good for: Main app sections, categories

3. Drawer Navigator

Side menu that slides in from the edge of the screen.

Characteristics:

- Hidden by default
- Opens from side gesture or button
- Good for: Settings, main menu, navigation options

Installing React Navigation

```
# Install core packages
npx expo install @react-navigation/native

# Install navigators (choose based on your needs)
npx expo install @react-navigation/native-stack
npx expo install @react-navigation/bottom-tabs
npx expo install @react-navigation/drawer

# Install dependencies
npx expo install react-native-screens react-native-safe-area-context
```

For drawer navigator, also install:

```
npx expo install react-native-gesture-handler react-native-reanimated
```

Navigation Container

All navigators must be wrapped in a **NavigationContainer**:

```
import { NavigationContainer } from '@react-navigation/native';

export default function App() {
  return (
    <NavigationContainer>
      { /* Your navigator here */ }
    </NavigationContainer>
  );
}
```

6. Navigating Between Screens

Stack Navigator

Basic Stack Navigation Setup

```
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';

const Stack = createNativeStackNavigator();

function HomeScreen({ navigation }) {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Home Screen</Text>
      <Button
        title="Go to Details"
        onPress={() => navigation.navigate('Details')}
      />
    </View>
  );
}

function DetailsScreen({ navigation }) {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Details Screen</Text>
      <Button
        title="Go Back"
        onPress={() => navigation.goBack()}
      />
    </View>
  );
}

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="Home" component={HomeScreen} />
        <Stack.Screen name="Details" component={DetailsScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

Navigation Methods

```
// Navigate to a screen
navigation.navigate('ScreenName');

// Go back to previous screen
navigation.goBack();

// Go back to specific screen in stack
navigation.navigate('ScreenName');

// Reset navigation stack
navigation.reset({
  index: 0,
  routes: [{ name: 'Home' }],
});

// Replace current screen
navigation.replace('ScreenName');
```

Tab Navigator

Basic Tab Navigation Setup

```
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import { NavigationContainer } from '@react-navigation/native';

const Tab = createBottomTabNavigator();

function HomeScreen() {
  return (
    <View style={styles.container}>
      <Text>Home Tab</Text>
    </View>
  );
}

function SettingsScreen() {
  return (
    <View style={styles.container}>
      <Text>Settings Tab</Text>
    </View>
  );
}

export default function App() {
  return (
    <NavigationContainer>
      <Tab.Navigator>
        <Tab.Screen name="Home" component={HomeScreen} />
        <Tab.Screen name="Settings" component={SettingsScreen} />
      </Tab.Navigator>
    </NavigationContainer>
  );
}
```

```

    </NavigationContainer>
  );
}

```

Tab Navigator with Icons

```

import { Ionicons } from '@expo/vector-icons';

export default function App() {
  return (
    <NavigationContainer>
      <Tab.Navigator>
        screenOptions={({ route }) => ({
          tabBarIcon: ({ focused, color, size }) => {
            let iconName;

            if (route.name === 'Home') {
              iconName = focused ? 'home' : 'home-outline';
            } else if (route.name === 'Settings') {
              iconName = focused ? 'settings' : 'settings-outline';
            }

            return <Ionicons name={iconName} size={size} color={color} />;
          },
          tabBarActiveTintColor: '#007AFF',
          tabBarInactiveTintColor: 'gray',
        })
      >
      <Tab.Screen name="Home" component={HomeScreen} />
      <Tab.Screen name="Settings" component={SettingsScreen} />
    </Tab.Navigator>
  </NavigationContainer>
);
}

```

Drawer Navigator

Basic Drawer Navigation Setup

```

import { createDrawerNavigator } from '@react-navigation/drawer';
import { NavigationContainer } from '@react-navigation/native';

const Drawer = createDrawerNavigator();

function HomeScreen({ navigation }) {
  return (
    <View style={styles.container}>
      <Text>Home Screen</Text>
    </View>
  );
}

```

```

        <Button
          title="Open Drawer"
          onPress={() => navigation.openDrawer()}
        />
      </View>
    );
  }

function ProfileScreen() {
  return (
    <View style={styles.container}>
      <Text>Profile Screen</Text>
    </View>
  );
}

export default function App() {
  return (
    <NavigationContainer>
      <Drawer.Navigator>
        <Drawer.Screen name="Home" component={HomeScreen} />
        <Drawer.Screen name="Profile" component={ProfileScreen} />
      </Drawer.Navigator>
    </NavigationContainer>
  );
}

```

Combining Navigators (Nested Navigation)

You can nest navigators to create complex navigation structures. This is common in real-world apps where you might have:

- A Stack Navigator for authentication flow
- A Tab Navigator for main app sections
- Stack Navigators within tabs for detailed navigation

Example: Stack → Tab → Stack Navigation

```

const Stack = createNativeStackNavigator();
const Tab = createBottomTabNavigator();
const ProfileStack = createNativeStackNavigator();

// Profile Stack Navigator (nested inside Tab)
function ProfileStackNavigator() {
  return (
    <ProfileStack.Navigator>
      <ProfileStack.Screen name="Profile" component={ProfileScreen} />
      <ProfileStack.Screen name="EditProfile" component={EditProfileScreen} />
      <ProfileStack.Screen name="Settings" component={SettingsScreen} />
    </ProfileStack.Navigator>
  );
}

```

```

}

// Tab Navigator (nested inside Main Stack)
function TabNavigator() {
  return (
    <Tab.Navigator>
      <Tab.Screen name="Home" component={HomeScreen} />
      <Tab.Screen name="Profile" component={ProfileStackNavigator} />
    </Tab.Navigator>
  );
}

// Main App with Stack Navigator
export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen
          name="Login"
          component={LoginScreen}
          options={{ headerShown: false }}
        />
        <Stack.Screen
          name="MainApp"
          component={TabNavigator}
          options={{ headerShown: false }}
        />
      </Stack.Navigator>
    </NavigationContainer>
  );
}

```

Navigation Structure

```

App (Stack Navigator)
├─ Login (Stack Screen)
└─ MainApp (Tab Navigator)
    ├─ Home (Tab Screen)
    ├─ Search (Tab Screen)
    └─ Profile (ProfileStack Navigator)
        ├─ Profile (Stack Screen)
        ├─ EditProfile (Stack Screen)
        └─ Settings (Stack Screen)

```

Navigating in Nested Navigators

When navigating in nested navigators, you can use dot notation:

```
// Navigate to a screen in nested stack
navigation.navigate('Profile', {
  screen: 'Settings',
  params: { userId: 123 }
});

// Or navigate directly if you're in the same navigator
navigation.navigate('EditProfile', { username: 'john' });
```

Best Practices for Nested Navigation

1. **Keep it Simple:** Don't nest too deeply (2-3 levels is usually enough)
2. **Clear Structure:** Each navigator should have a clear purpose
3. **Header Management:** Use `headerShown: false` appropriately to avoid duplicate headers
4. **Initial Routes:** Set `initialRouteName` for nested navigators when needed
5. **Data Passing:** Use navigation params to pass data between screens in nested navigators

7. Passing Data Using Navigation

You can pass data between screens using navigation parameters (route params).

Passing Data to a Screen

```
// HomeScreen - Passing data
function HomeScreen({ navigation }) {
  const user = {
    id: 1,
    name: 'John Doe',
    email: 'john@example.com',
  };

  return (
    <View style={styles.container}>
      <Button
        title="View Profile"
        onPress={() => navigation.navigate('Profile', { user })}
      />
    </View>
  );
}

// ProfileScreen - Receiving data
function ProfileScreen({ route, navigation }) {
  const { user } = route.params;

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Profile</Text>
    </View>
  );
}
```

```
        <Text>Name: {user.name}</Text>
        <Text>Email: {user.email}</Text>
    </View>
  );
}
```

Handling Missing Parameters

Always check if parameters exist:

```
function DetailsScreen({ route }) {
  const { itemId, otherParam } = route.params || {};

  return (
    <View style={styles.container}>
      <Text>Item ID: {itemId || 'No ID provided'}</Text>
      <Text>Other Param: {otherParam || 'No other param'}</Text>
    </View>
  );
}
```

Updating Screen Options Based on Params

```
function ProfileScreen({ route, navigation }) {
  const { userName } = route.params || {};

  React.useEffect(() => {
    if (userName) {
      navigation.setOptions({
        title: `${userName}'s Profile`,
      });
    }
  }, [userName, navigation]);

  return (
    <View style={styles.container}>
      <Text>Profile for {userName}</Text>
    </View>
  );
}
```

Passing Data Back to Previous Screen

Using Callbacks

```
// HomeScreen - Set up callback
function HomeScreen({ navigation }) {
  const handleSelectItem = (itemId) => {
    // Navigate with callback
    navigation.navigate('ItemSelection', {
      onSelect: (selectedItem) => {
        console.log('Selected item:', selectedItem);
        // Use the selected item
      },
    });
  };

  return (
    <Button
      title="Select Item"
      onPress={() => handleSelectItem(123)}
    />
  );
}

// ItemSelectionScreen - Use callback
function ItemSelectionScreen({ route, navigation }) {
  const { onSelect } = route.params;

  const handleSelect = () => {
    const selectedItem = { id: 1, name: 'Item 1' };
    onSelect(selectedItem);
    navigation.goBack();
  };

  return (
    <Button title="Select This Item" onPress={handleSelect} />
  );
}
```

Using navigation.setParams()

```
function EditScreen({ route, navigation }) {
  const [editedData, setEditedData] = useState(route.params?.data || {});

  const saveData = () => {
    // Update params in previous screen
    navigation.setParams({ data: editedData });
    navigation.goBack();
  };

  return (
    <View>
      <TextInput
        value={editedData.name}
      />
    </View>
  );
}
```

```
      onChangeText={({text}) => setEditedData({ ...editedData, name: text })}
    />
    <Button title="Save" onPress={saveData} />
  </View>
);
}
```

Best Practices for Navigation

1. **Type Safety:** Use TypeScript or PropTypes for navigation params
2. **Default Values:** Always provide default values for optional params
3. **Deep Linking:** Consider deep linking when designing navigation structure
4. **Performance:** Use React.memo for screens that re-render frequently
5. **Navigation State:** Don't store large amounts of data in navigation params; use state management libraries for complex global state
6. **Params for Screen-Specific Data:** Use navigation params for data that's only needed by the target screen

Summary

This tutorial covered essential concepts for building interactive React Native applications:

- **Props and States:** Understanding data flow in React components
- **React Hooks (useState):** Modern way to manage component state
- **User Input & Forms:** Handling text input and form validation
- **Touch Handling:** Responding to user interactions
- **Navigation:** Implementing Stack, Tab, and Drawer navigators
- **Screen Navigation:** Moving between different screens
- **Data Passing:** Sharing data between screens using navigation params

Next Steps

- Practice building forms with validation
- Create a multi-screen app with navigation
- Explore advanced navigation patterns (nested navigators)
- Learn about other React hooks (useEffect, useCallback, useMemo)
- Study React Navigation documentation: <https://reactnavigation.org>
- Explore state management libraries (Redux, Zustand) for complex apps

Additional Resources

- [React Navigation Documentation](#)
- [React Hooks Documentation](#)
- [React Native TextInput](#)
- [React Native Touchable Components](#)
- [Safe Area Context](#)